YANTRA

Complete Project Developer Documentation

Your Networked Tactical Response Assistant

Architecture • Installation • Configuration • Voice • Automation • APIs • Testing

Prepared from the current project implementation

Table of Contents

Table of Contents 2

YANTRA 4

Contents . . . 4
Capabilities . . . 4
Architecture . . . 5
Main runtime components . . . 6
Repository layout . . . 6
Requirements . . . 6
Operating system . . . 6
Python . . . 6
External services . . . 7
Installation . . . 7
1. Select or create a Python environment . . . 7
2. Install assistant dependencies . . . 7
3. Install the Playwright browser . . . 7
4. Configure environment variables . . . 7
Configuration . . . 7
Running the application . . . 8
Using the dashboard . . . 8
Typed commands . . . 9
Voice mode . . . 9
Text-command flow . . . 9
Live voice flow . . . 9
Audio backend selection . . . 10
Audio backends and troubleshooting . . . 10
Underflow . . . 10
Overflow . . . 10
PortAudio / WDM-KS startup errors . . . 10
Automation tools . . . 11
HTTP and Socket.IO interfaces . . . 11
HTTP routes . . . 11
Socket.IO events . . . 11
File uploads and privacy . . . 12
Memory and proactive features . . . 12

Testing 12
Security and operational guidance 13
Troubleshooting 13
ModuleNotFoundError 13
Port 5000 is already in use 13
Gemini Live will not connect 13
Typed requests do not respond 14
Browser automation fails 14
The UI shows old code 14
Development notes 14
License and third-party services 14

YANTRA

Your Networked Tactical Response Assistant

YANTRA is a Windows desktop assistant with a local web dashboard. It combines Gemini and Groq language models with voice interaction, native audio streaming, screen and webcam capture, browser automation, file operations, system monitoring, reminders, web search, and other desktop actions.

The application is designed to run locally. The dashboard binds to `127.0.0.1` by default, while selected requests and uploaded files may be sent to external AI providers when those features are used.

Contents

- Capabilities
- Architecture
- Repository layout
- Requirements
- Installation
- Configuration
- Running the application
- Using the dashboard
- Text-command flow
- Live voice flow
- Audio backends and troubleshooting
- Automation tools
- HTTP and Socket.IO interfaces
- File uploads and privacy
- Memory and proactive features
- Testing
- Security and operational guidance
- Troubleshooting
- Development notes

Capabilities

- Local Flask + Socket.IO dashboard.
- Typed assistant requests with Gemini-first routing and Groq fallback.
- Gemini Live native-audio voice sessions with interruption support.
- WASAPI audio on Windows, with DirectSound fallback when WASAPI is unavailable or blocked.
- Automatic audio-stream startup retries.
- Gemini file analysis for supported uploads.
- Registered desktop tools for:
- Opening and closing applications.

- Browser navigation and web search.
- Reading, writing, listing, and opening files.
- Mouse, keyboard, screenshots, and active-window inspection.
- Screen and webcam capture.
- Computer settings and system information.
- YouTube, weather, flight, messaging, reminders, game updates, and code assistance.
- Real-time CPU, memory, disk, network, process, and activity information.
- Saved conversation memory and optional saved facts.
- Optional proactive monitoring and context-aware check-ins.
- Local TTS fallback through `pyttsx3`.
- Whisper-based non-Live transcription support.

Architecture

Main runtime components

Component	Responsibility
ui_server.py	Flask app, Socket.IO events, dashboard state, uploads, voice worker, monitoring broadcasts
gemini_assistant.py	Gemini/Groq configuration, typed assistant requests, Live configuration, file analysis
live_client.py	Live-session state and audio-output queue abstractions
execute.py	Entry point for executing an assistant action
tools/register.py	Local tool registry and dispatch
`backend/tools/`	Core local tools
`backend/mark_actions/`	Extended automation modules
memory.py	Conversation memory persistence
`backend/ui/templates/index.html`	Dashboard markup
`backend/ui/static/css/yantra.css`	Dashboard styling and network map
`backend/ui/static/js/yantra.js`	Dashboard behavior and Socket.IO client

Repository layout

```
.
├── backend/
│   ├── .env.example
│   ├── requirements-assistant.txt
│   ├── ui_server.py
│   ├── gemini_assistant.py
│   ├── live_client.py
│   ├── execute.py
│   ├── memory.py
│   ├── tts.py
│   ├── transcribe.py
│   ├── tools/
│   ├── mark_actions/
│   ├── tests/
│   └── ui/
├── venv/                # project Python environment, if present
├── .venv/               # optional alternate environment
└── README.md
```

The `venv` and `.venv` directories are local environments and should not be committed. Use the environment that points to an installed Python runtime; a stale environment may reference a Python executable that no longer exists.

Requirements

Operating system

The project is primarily implemented for Windows. Several tools use Windows APIs or desktop packages, including `pywin32`, `pywinauto`, `pycaw`, `win10toast`, and Windows audio backends.

Python

Use Python 3.11 or a compatible version supported by the installed dependencies. The repository's working environment has been validated with Python 3.11.

External services

The following are optional by feature:

- Google AI Studio / Gemini API key:
- Typed Gemini requests.
- Gemini Live voice.
- Gemini file analysis.
- Groq API key:
- Text fallback.
- Compound web search, when enabled for the configured account.
- Playwright Chromium:
- Browser automation features.

Installation

Open PowerShell in the repository root.

1. Select or create a Python environment

If the included environment works:

```
.\venv\Scripts\Activate.ps1
```

If a new environment is needed:

```
py -3.11 -m venv venv
.\venv\Scripts\Activate.ps1
```

2. Install assistant dependencies

```
python -m pip install --upgrade pip
python -m pip install -r backend\requirements-assistant.txt
```

3. Install the Playwright browser

Only required for browser automation:

```
python -m playwright install chromium
```

4. Configure environment variables

Copy the example file:

```
Copy-Item backend\.env.example backend\.env
```

Edit `backend\.env` and add the provider keys described below. Never commit this file or share its contents.

Configuration

Configuration is loaded from `backend\.env` and the process environment.

Variable	Purpose	Typical value
`GEMINI_API_KEY`	Gemini authentication	Provider-issued secret
`GEMINI_MODEL`	Typed Gemini model	`gemini-3.8-flash` or an available model
`GEMINI_LIVE_MODEL`	Gemini Live native-audio model	`models/gemini-2.5-flash-native-audio-preview-12-2025`
`GROQ_API_KEY`	Groq authentication	Provider-issued secret

Variable	Purpose	Typical value
`GROQ_MODEL`	Groq text fallback model	`openai/gpt-oss-20b`
`GROQ_SEARCH_MODEL`	Groq search model	`compound-beta`
`YANTRA_PORT`	Local dashboard port	`5000`
`YANTRA_SECRET_KEY`	Optional Flask secret	Random private value

Do not place API keys in HTML, JavaScript, screenshots, source control, activity logs, or conversation memory.

Running the application

From the repository root:

```
python backend\ui_server.py
```

Open:

```
http://127.0.0.1:5000
```

The server prints the URL during startup. It binds to localhost only and runs with the Flask-SocketIO development server.

To use a different port:

```
$env:YANTRA_PORT = "5100"
python backend\ui_server.py
```

To verify the server:

```
Invoke-WebRequest -UseBasicParsing http://127.0.0.1:5000/
```

An HTTP `200` response confirms that the dashboard route is reachable.

Using the dashboard

The dashboard provides:

- System status cards.
- Core assistant status.
- Gemini Live and Groq provider status.
- Automation-tool count.
- Activity log with source, tag, time, and message.
- Network map and active-connection count.
- Real-time performance graph.
- Top-process list.
- Memory and disk information.
- Text command entry.
- Microphone / voice mode controls.
- File upload and analysis.
- Vision/screen capture controls.
- Memory clearing and mode toggles.

Typed commands

Enter a request in the command panel.

Submit the request.

The server runs the assistant request in a worker thread.

The model may return text, tool calls, or both.

Tool calls are validated and dispatched through the local registry.

Results are sent back to the dashboard and activity log.

Voice mode

Confirm a microphone and speaker are available.

Select the voice or microphone control.

Allow microphone access if prompted.

The server connects to Gemini Live.

Native audio is sent to the local output device.

Speech interruption clears pending assistant audio and resumes listening.

Stop voice mode to close the Live session and local streams.

`AUTO LISTEN` is an explicit opt-in browser preference. It should not be enabled unless automatic microphone activation is wanted.

Text-command flow

The typed path is:

```
Socket.IO text_command
    -> ui_server.py worker
    -> gemini_assistant.run_assistant
    -> Gemini request
    -> Groq fallback when configured/needed
    -> tool call through execute.py
    -> tools/register.py
    -> activity/result events
```

The local execution layer returns structured results with a success flag and user-facing message. Unsupported actions are rejected rather than silently ignored.

Live voice flow

The Live worker in `ui_server.py` maintains:

- A Gemini Live connection.
- An asyncio input queue for microphone chunks.
- A bounded output queue for generated PCM audio.
- A speaker callback that consumes output audio.
- Input and output transcription handling.
- Session resumption information.
- Interruption and reconnect state.
- Proactive-engine checks when the session is idle.

Audio backend selection

On Windows, the application tries backends in this order:

WASAPI

- Matches the current default microphone and speaker by name.
- Uses shared mode.
- Enables automatic sample-rate conversion.

DirectSound fallback

- Uses matching DirectSound devices when WASAPI is blocked by a recorder, driver, or endpoint state.

Default sounddevice configuration

- Used only when host-specific device discovery is unavailable.

Audio startup is retried up to three times for each backend. Failed streams are stopped and closed before the next attempt. The activity log includes the backend, attempt number, exception type, and full PortAudio message.

Audio backends and troubleshooting

Underflow

`Speaker status: output underflow` means the speaker callback requested audio but the output queue did not contain enough PCM data. The callback fills the missing portion with silence.

Common causes:

- Network jitter.
- CPU contention.
- A recorder or other application competing for the output device.
- Audio driver timing problems.

Overflow

`Microphone status: input overflow` means microphone frames arrived faster than the callback/event loop could process them. Some input frames were discarded.

Common causes:

- CPU contention.
- An overloaded event loop.
- A recorder competing for the microphone.
- Driver or host-API scheduling delays.

PortAudio / WDM-KS startup errors

If logs contain `WdmSyncIoctl`, `GLE = 0x00000490`, or `PaErrorCode -9999`:

Close or reconfigure screen recorders, Teams, Zoom, Discord, and browser microphone capture.

Start the recorder first, then start YANTRA.

Disable exclusive control in Windows device properties.

Confirm the Realtek microphone and speaker are selected as Windows defaults.

Disconnect and reconnect Bluetooth audio devices.

Use headphones or a separate USB microphone if the recorder must capture system audio.

Check that Windows microphone permissions are enabled.

The application attempts DirectSound automatically after WASAPI failures. The log should show:

```
DirectSound fallback audio enabled
```

Automation tools

Tools are registered in tools/register.py. Core groups include:

Group	Examples
Applications	Open/close applications, create a blank Excel workbook
Browser	Open websites, web search
System	System status and resource information
Files	List, read, write, and open files
Computer	Mouse, keyboard, hotkeys, active window
Vision	Screen and webcam capture
Extended automation	Weather, flights, YouTube, reminders, messaging, games, settings, code helper, monitoring

The exact tool count is calculated at runtime. Run:

```
python backend\tools\register.py
```

Tool execution accepts a dictionary command. The dispatcher supports `_tool_name` for model-produced calls and normalizes it to the internal action name. Failures are returned as structured error results and recorded in the activity log.

HTTP and Socket.IO interfaces

HTTP routes

Route	Method	Purpose
`/`	`GET`	Render the dashboard
`/api/upload`	`POST`	Upload a file for Gemini analysis

Uploads are limited to 25 MB by the Flask configuration.

Socket.IO events

The dashboard uses events including:

Event	Direction	Purpose
`connect` / `disconnect`	Browser -> server	Track dashboard clients
`text_command`	Browser -> server	Submit a typed assistant request
`start_voice`	Browser -> server	Start Gemini Live
`stop_voice`	Browser -> server	Stop Gemini Live
`interrupt`	Browser -> server	Stop current assistant audio

Event	Direction	Purpose
`clear_memory`	Browser -> server	Clear conversation memory
`set_vision_mode`	Browser -> server	Enable or disable vision mode
`status_update`	Server -> browser	Assistant state
`voice_state`	Server -> browser	Voice lifecycle state
`activity_update`	Server -> browser	Activity-log entries
`metrics_update`	Server -> browser	System metrics
`assistant_result`	Server -> browser	Assistant response
`assistant_error`	Server -> browser	User-visible failure
`waveform_active`	Server -> browser	Audio activity indicator

The exact payload shape is defined by the event handlers in ui_server.py and the client handlers in yantra.js.

File uploads and privacy

Uploaded files may be sent to Gemini for analysis. The server uses temporary local copies and attempts to remove temporary files after processing. Where supported by the provider, uploaded provider-side files are also deleted after processing.

Do not upload confidential, regulated, or private material unless the data-sharing implications of the configured provider are acceptable.

Memory and proactive features

- Conversation memory is managed by memory.py.
- Saved facts and monitored topics are stored under `backend/mark_actions/data/` when enabled.
- Monitoring is opt-in.
- Financial and crypto topics are blocked by the proactive-monitoring rules.
- Memory and monitoring data should remain local and excluded from source control.

Testing

Run the focused backend tests from the repository root:

```
.\venv\Scripts\python.exe -m unittest backend.tests.test_ui_server backend.tests.test_live_client
```

The tests cover:

- Dashboard loading.
- Port configuration.
- Provider configuration.
- Voice start guards.
- Transcript merging.
- Live-session lifecycle.
- Audio stream mocking.
- Tool dispatch behavior.
- Typed TTS behavior.

For a broader test run:

```
.\venv\Scripts\python.exe -m unittest discover -s backend\tests -p "test_*.py"
```

Before submitting changes:

Run the focused tests for the changed component.

Check Python syntax.

Run `git diff --check`.

Start the dashboard and verify the main route.

For UI changes, reload the dashboard and inspect the affected panel.

Security and operational guidance

- Keep `backend\.env` private.
- Never commit API keys, tokens, or private uploaded files.
- Keep the server bound to `127.0.0.1` unless a deliberate deployment design adds authentication and transport security.
- Review every desktop/file/browser tool before enabling it in a less-trusted environment.
- Treat model-generated tool arguments as untrusted input.
- Avoid running the assistant with administrator privileges.
- Do not upload sensitive files to external model providers without approval.
- Monitor provider quotas and billing.
- Do not expose the Flask development server directly to the public internet.

Troubleshooting

`ModuleNotFoundError`

Use the same environment for installation and execution:

```
.\venv\Scripts\python.exe -m pip install -r backend\requirements-assistant.txt
.\venv\Scripts\python.exe backend\ui_server.py
```

Port 5000 is already in use

Find the process:

```
Get-NetTCPConnection -LocalPort 5000
```

Alternatively select another port:

```
$env:YANTRA_PORT = "5100"
.\venv\Scripts\python.exe backend\ui_server.py
```

Gemini Live will not connect

Check:

- `GEMINI_API_KEY` is present.
- `GEMINI_LIVE_MODEL` is available to the configured project.
- The project has Live API quota.
- The activity log contains the provider error.
- The machine has a working microphone and speaker.

Typed requests do not respond

Check:

- Gemini credentials and model configuration.
- Groq credentials if fallback is expected.
- The activity log for tool or provider errors.
- That the dashboard is connected to the same server process being edited.

Browser automation fails

Install Chromium:

```
.\venv\Scripts\python.exe -m playwright install chromium
```

The UI shows old code

Restart `ui_server.py` after backend or template changes, then hard-refresh the browser. Flask template/static caching can otherwise make an older process or browser tab appear unchanged.

Development notes

- The dashboard entry point is backend/ui_server.py.
- The server uses `use_reloader=False`; restart it manually after source changes.
- The map is an inline SVG data URL in yantra.css.
- The frontend is served from `backend/ui/templates/` and `backend/ui/static/`.
- Local runtime data, uploads, and assistant memory should not be treated as source files.
- Keep changes focused and preserve the existing event names and structured result format.

License and third-party services

This repository uses third-party Python packages and external AI services. Review the applicable licenses, provider terms, quotas, and privacy policies before distributing or deploying the project.